Scanned Code Report

AUDITAGENT

Powered by NETHERMIND SECURITY

Code Info

Auditor Scan

Scan ID
4

Date
February 10, 2026

Organization
chipi-pay

Repository
sessions-smart-contract

Branch
main

Commit Hash
9be9629b...95d4dfd9

Contracts in scope

src/account.cairo
src/lib.cairo

Code Statistics

Findings
0

Contracts Scanned
2

Lines of Code
893

Findings Summary

Code Summary

This protocol implements an advanced smart contract account on Starknet, built upon OpenZeppelin's standard components for core account functionality, upgradeability, and introspection. The primary feature of this account is its sophisticated session key management system, designed to provide granular, temporary permissions for dApp interactions without exposing the owner's primary key.

Session keys are temporary cryptographic keys that can be authorized by the account owner to perform a limited set of actions. Each session key is configured with specific constraints:
- **Expiration Time:** A `valid_until` timestamp after which the key is no longer valid.
- **Call Limit:** A `max_calls` counter that restricts the total number of transactions the key can sign.
- **Function Whitelist:** An `allowed_entrypoints` list that specifies exactly which contract functions (identified by their selectors) the session key is permitted to call.

The account's validation logic is customized to handle multiple signature schemes. When a transaction is submitted, the `__validate__` function checks the signature format to determine the signing authority:
1. **Owner Signature:** A standard 2-element signature (`[r, s]`) is validated against the account's main public key, providing full control.
2. **Session Key Signature:** A 4-element signature (`[session_pubkey, r, s, valid_until]`) triggers the session key validation flow. The contract verifies the key's existence, expiration, call limit, and ensures the transaction's calls are compliant with the predefined function whitelist.

To prevent privilege escalation, the session key mechanism includes robust security measures. There is a hardcoded blocklist of administrative functions (e.g., `upgrade`, `set_public_key`, session management) that session keys can never call. Furthermore, if a session key is configured with an open whitelist (allowing any function call), it is explicitly blocked from making any calls to the account contract itself, thus preventing it from accessing any owner-only functions.

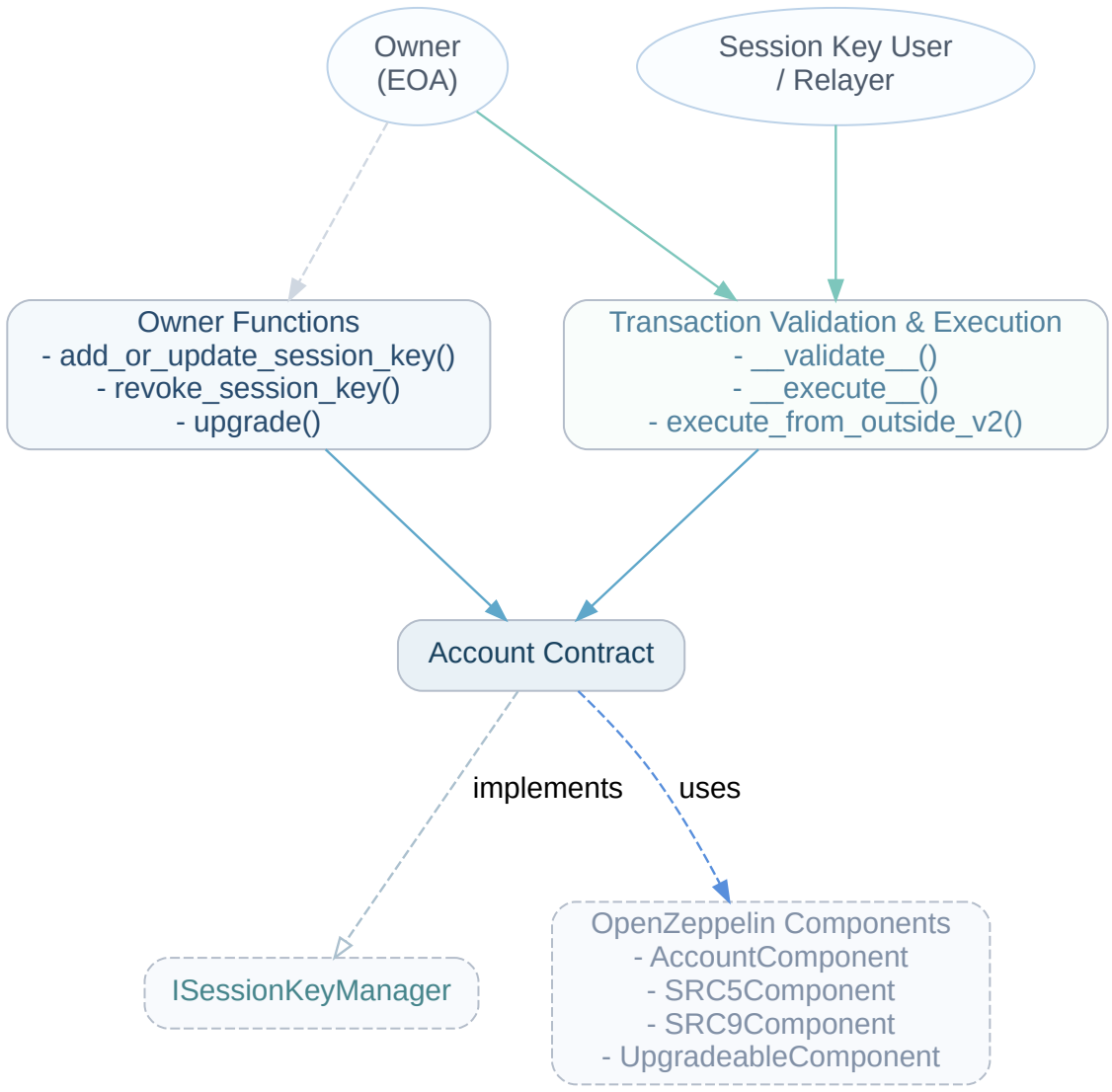
The contract also supports meta-transactions through a custom implementation of SNIP-9 (`execute_from_outside_v2`), allowing third-party relayers to submit transactions on behalf of the user. This implementation fully integrates the session key validation logic, ensuring that even relayed transactions adhere to the specified permissions.

Entry Points & Actors

The main state-modifying entry points of the protocol are:

- `__validate__(calls)`: This function is called by the Starknet sequencer to validate a transaction. It checks the signature of the `Account Owner` or a `Session Key Holder`. If a valid session key is used, it consumes one of the key's available calls.
- `__execute__(calls)`: Following a successful validation, this function is called by the sequencer to execute the batch of calls in the transaction. It is initiated by either the `Account Owner` or a `Session Key Holder`.
- `execute_from_outside_v2(outside_execution, signature)`: Allows a `Relayer` to submit a transaction on behalf of the `Account Owner` or a `Session Key Holder`. This function validates the time bounds, consumes a nonce to prevent replays, validates the signature, and executes the specified calls. It also enforces all session key permissions if a session key signature is provided.

Code Diagram



Disclaimer

Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.